TECHNICAL REPORT

Survival Model Evaluation: synthetic

Fitted with the survival-analysis-pipeline, on synthetic data with known ground truth

Source data	`data/strategies.csv`, drawn at seed 7
Rows	5,000 (93.2% with the ending observed, 6.8% censored)
Start dates	2021-07-01 to 2026-06-01
Evaluation	5 expanding-window temporal folds
Source of figures	`reports/synthetic/metrics.json`, regenerated by the command in section 9

---

1. Summary

This report evaluates two survival models fitted to synthetic data drawn at seed 7. It holds 5,000 rows, each observed from its start date. 93.2% have observed endings and 6.8% are censored, still running when observation stopped. Median observed duration, censored rows included, is 91 days, the scale on which every horizon and prediction below sits. The models predict, from what was on file at the start date, how long each row survives.

When evaluating the two models' performance, the apples-to-apples comparison is the fold-mean concordance, the share of row pairs a ranking orders correctly where 0.500 is a coin flip. The pipeline uses walkforward validation. Each of the 5 temporal folds trains both models on one stretch of history and tests them on the next, and the 5 scores average. On concordance the XGBoost accelerated-failure-time (AFT) model scores 0.781 and the Cox proportional hazards baseline 0.781. The two models tie at the printed precision.

To attach an uncertainty range, which says how far the score could reasonably move, the AFT model is also scored with all 3,000 out-of-time test rows pooled into one list. That concordance is 0.781, with a 95% bootstrap interval of 0.773 to 0.790. Section 5 explains the two figures and how to read the interval.

Little of the pooled score is `asset_class` group membership, where `asset_class` is the grouping column this run was given. Ranking rows by their group's average prediction alone scores 0.538, and comparing only rows inside the same group scores 0.779, clear of the coin flip. Section 5 gives the decomposition.

An oracle ranking, defined in section 5, bounds every model at 0.820.

Both models are saved in one bundle, which records the Cox baseline as recommended for scoring new rows, on a near-tie margin.

> All results are synthetic. The run validates the pipeline and asserts nothing about live data.

2. Motivation

Analyst notes:

This run exists to validate the pipeline against a known answer. The population is synthetic, 5,000 trading strategies with the metadata a search records on the day each one is deployed, and the generator installs the structure that drives their lifetimes. So the best achievable ranking is computable, and the model's attributions can be checked against the mechanisms actually installed. The strategy framing is vocabulary, not a claim. Nothing here is evidence about any real book. The question the run answers is whether the pipeline recovers structure that is known to be there, and how far short of the ceiling it lands.

On a real book, death would be a bookkeeping event, the date a strategy stops clearing the book's retention rule, and the lifetimes a model learns there are partly a property of that rule. The generator abstracts this to a lifetime drawn at birth.

3. Data

Durations are measured in days.

Left truncation, where a row was already running when the source's records begin, leaves a recorded start that is not the true start. This pipeline has no delayed-entry handling, so it cannot take a row that entered observation partway through its life, and those rows must be excluded during preparation. Whether they were, and how, is recorded in the dataset's own documentation.

Analyst notes:

Two generator settings matter for interpreting the results. 6% of rows are administratively retired independent of performance, which installs the censoring the evaluation must handle correctly. And lifetimes are drawn log-normally around what their drivers predict, at noise scale 0.55, the irreducible noise that keeps even the oracle in section 5 short of a perfect score.

The generator intentionally installs detectable structure in the data, so that recovering it is a test rather than an interpretation. Each of the six feature families shifts log survival time by a fixed amount, microstructure the most fragile and value-carry the most durable. Among the four asset classes, crypto is built to decay fastest.

Censoring concentrates among recently discovered strategies, which have had the least time to fail before the observation cutoff, so the final fold's test block is far more censored than the population overall.

Every lifetime here is drawn at birth and observed from its true start, so the left-truncation fault described previously cannot arise in this population.

Figure 1 shows Kaplan-Meier survival curves by `asset_class`, the coarsest structure in the outcome before any model is fitted.

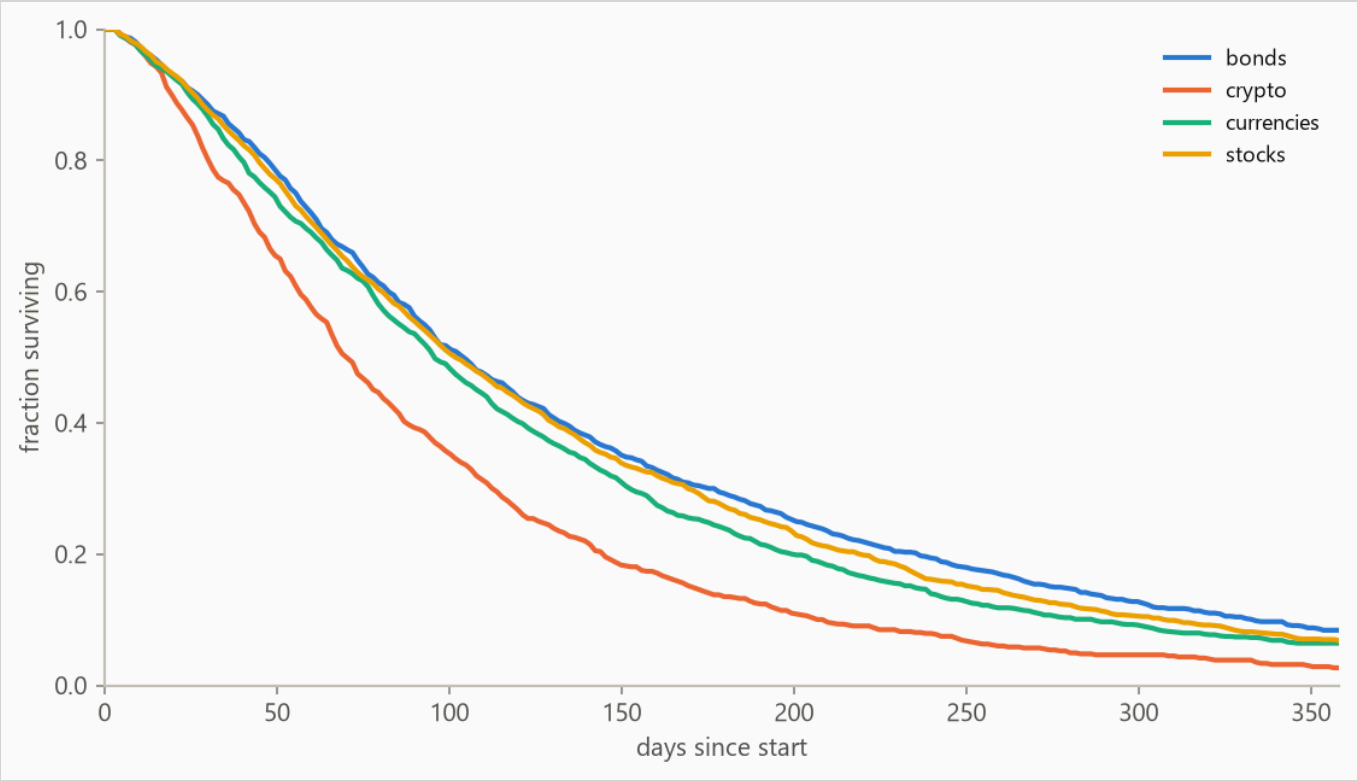

Figure 1. Kaplan-Meier survival curves by `asset_class`: the fraction of each group still running at each age, with censored rows counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

4. Method

4.1 Model class

The pipeline fits two models on every run. The first is a boosted-tree accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending tells the model the exact lifetime, and a censored row tells it only "at least this long", so every row contributes. It predicts each row's median survival time in days, and a log-normal curve around that median, whose width is fitted once and shared by every row, gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features to answer whether the boosted model was necessary. Its coefficients are fitted under a penalty that pulls them toward no effect, which holds the fit steady when features move together but leaves the uncertainty intervals drawn around those coefficients approximate. The run recommends whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every row follows the same survival curve run on a faster or slower clock, so features change how fast that clock runs, never the curve's shape. The Cox model makes no assumption about that shape at all, estimating the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

4.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by start date. The earliest 40% of rows is set aside as burn-in and never tested. Each fold trains on every row started before its split date and tests on the next block. A split date can only fall on a date the data contains, so the first fold trains on 1,999 rows, the burn-in rounded to a date boundary (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future.

4.3 Selection and calibration

The boosted model's hyperparameters are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether rows are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's log-normal curve, one unitless number shared by every row. A larger scale spreads the model's probability over a wider range of lifetimes. Two values of it appear in this run. The first, 0.6, is the setting used while training, where it shapes how the medians are fitted. The second, 0.64, is measured afterwards. With the fitted medians held fixed, it is the width that best matches the held-out outcomes on the training window's most recent stretch, and it is carried to the model refitted on the full window. The training value answers which width trains the best medians. The measured value answers which width matches the outcomes those medians actually got. The two need not agree, and every probability the boosted model reports here uses the measured value.

5. Results

5.1 Discrimination

Table 1. Concordance index by model, pooled over 3,000 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip, and Harrell's C is the standard estimator of it. Fold-mean rows score each of the 5 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
Oracle ranking on latent log-time (ceiling)	0.820	not resampled
XGBoost AFT (pooled)	**0.781**	**0.773 to 0.790**
XGBoost AFT (fold mean)	0.781	not resampled
Cox proportional hazards (fold mean)	0.781	not resampled

Each fold fits its own models. Because the AFT model predicts a survival time in days, and days is a universal unit that exists outside of an individual fold model, its predictions can be pooled into one list while Cox proportional hazards models cannot. A Cox model predicts a hazard score, the risk of ending at any given moment, relative to the average row in the window it was trained on, not universal time units. As a result, Cox

scores from different folds cannot be compared in one list. For a fair comparison, the two models must be compared on fold mean concordance, which is each fold scored on its own and the 5 scores averaged.

The pooled row answers a different question. It scores all test rows in one list, which is enough data to attach an uncertainty range as seen in Table 1. The interval is the range the score stayed inside for 95% of resamples of the test rows. A narrow interval means the score is precise on this test set. And an interval that sits wholly above 0.500 means the model beats a coin flip even at its low end.

It is important to determine whether the model does more than recognize which group a row belongs to. In order to assess this, we split the pooled concordance by `asset_class`. Re-ranking rows by their group's average replaces every row in a group with the same score, so comparisons only ever run between groups. Evaluating the pooled concordance on those re-ranked rows gives a score of 53.8%. The boosted model scores each row individually instead, which orders rows inside a group. Inside a group this ranking is correct 77.9% of the time, averaged over the 4 groups large enough to score (at least 50 rows and 10 observed endings), each weighted by its number of comparable pairs. When the group-average score matches or beats the model's own pooled score, the model's ranking comes from telling groups apart. When the within-group score sits well above 0.500, the model also orders rows inside a group, which is the part a lookup table of group averages cannot do.

The oracle ranking in Table 1 orders rows by the latent log-time the generator actually used. Nothing is fitted and the ranking predates the noise draw, so its 0.820 bounds every model. The gap to a perfect score is installed noise. A suite test asserts, on a generated run, that no model outscores it, since beating perfect information indicates a leak.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX	ORACLE
1	2023-06-22	1,999	600	2%	0.780	0.783	0.824
2	2024-02-03	2,600	600	2%	0.768	0.771	0.808
3	2024-09-01	3,199	600	2%	0.768	0.765	0.805
4	2025-04-03	3,800	600	9%	0.772	0.770	0.811
5	2025-11-03	4,399	600	39%	0.817	0.816	0.846

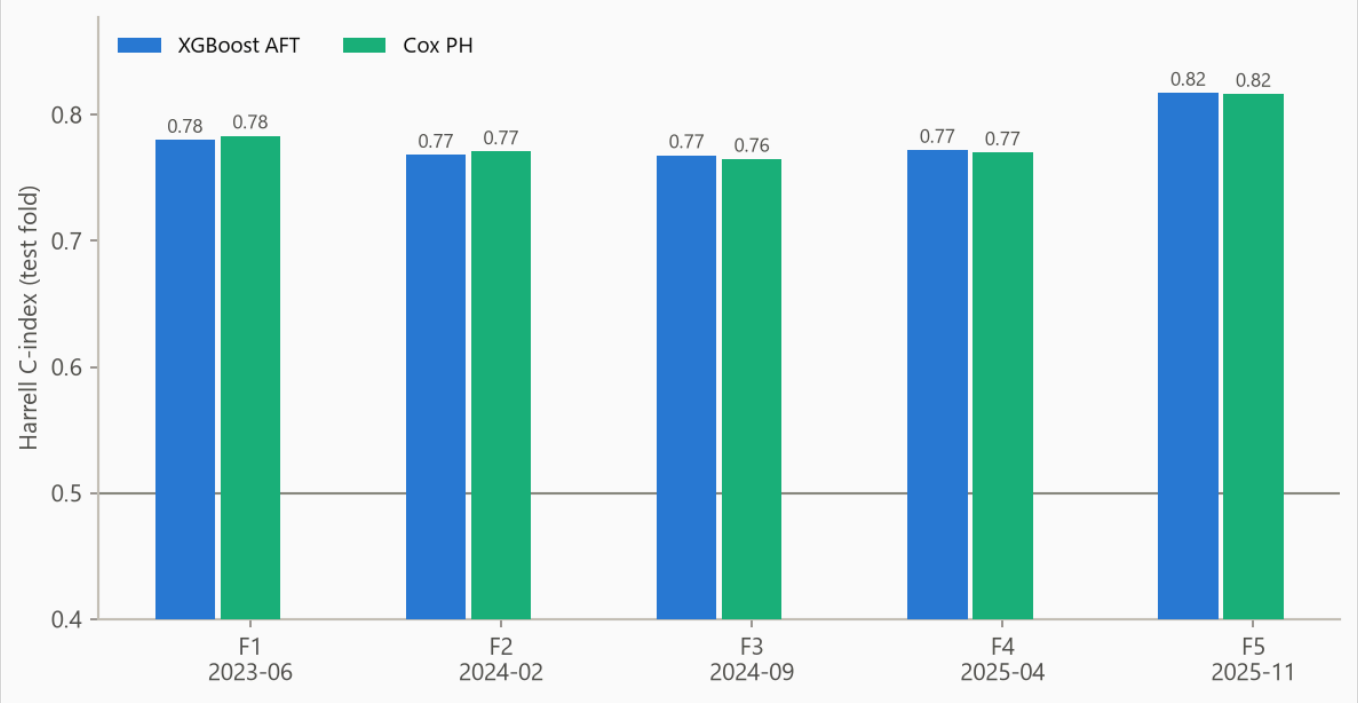

Figure 2. Concordance by fold. The boosted model spans 0.768 to 0.817. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

5.2 Calibration

A majority of this report focuses on concordance. But concordance grades only whether models produce the correct ordering when comparing two rows. It cannot be used to assess the absolute accuracy of the predicted probabilities. To assess the absolute accuracy, we use the Brier score, the mean squared error of a probability forecast (lower is better). Censored rows can bias the Brier score, so we use the inverse probability of censoring weighting (IPCW) to compensate. At each horizon in Table 3, the models predict each row's chance of still running that length of time. The no-skill forecast predicts the same chance for every row, the share of the whole population still running at that horizon, estimated with the Kaplan-Meier method so censored rows count for as long as they were observed.

Table 3. IPCW Brier score by horizon. Lower is better, and the no-skill column is the score to beat, so a model above it adds nothing at that horizon.

HORIZON	AFT	COX	NO-SKILL FORECAST
90 days	0.148	0.150	0.249
180 days	0.132	0.132	0.182
365 days	0.051	0.051	0.056

Figure 3 plots predicted against observed survival at 180 days for both models. Points falling on both sides of the diagonal are noise. Points falling consistently on one side are bias in that direction.

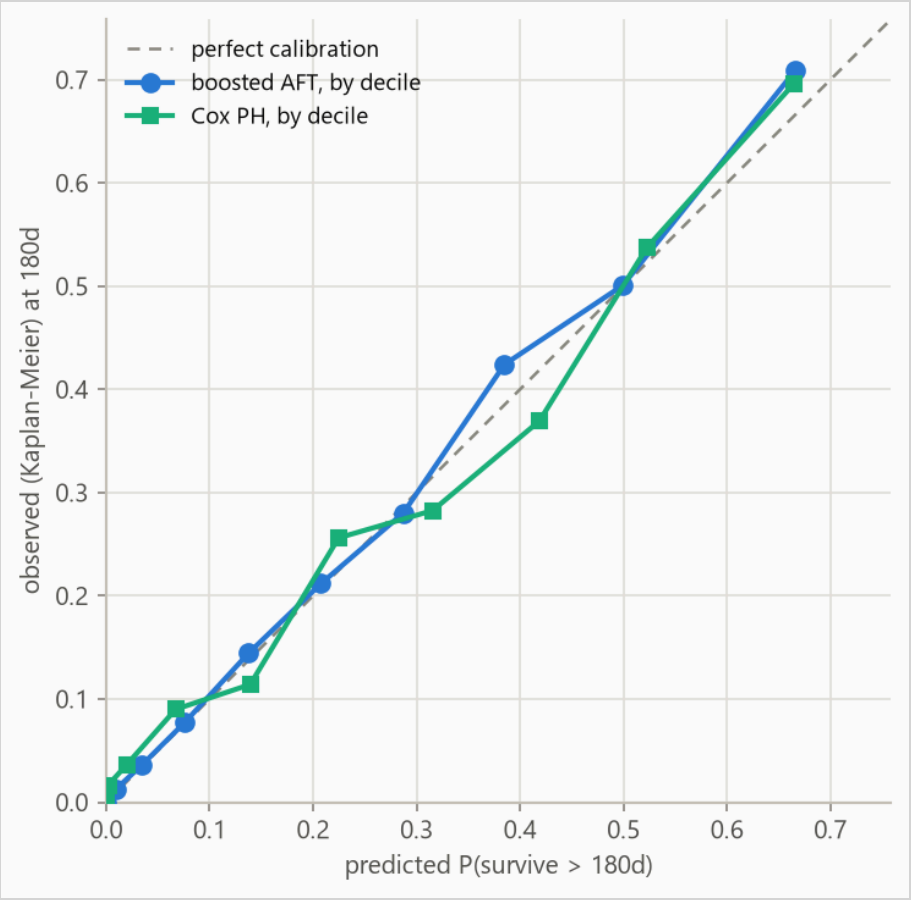

Figure 3. Predicted against observed survival at 180 days for both models, each binned on its own predicted deciles. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.043 in the boosted model's decile 10 and 0.049 in the Cox baseline's decile 8. Deviations are probabilities. Deciles are cut on predicted value and hold 300 rows each. In small or heavily censored deciles the observed value carries more uncertainty than three decimals suggest.

6. Feature analysis

6.1 The boosted model

The scores above grade how well each model ranks rows and how accurate its probabilities are, not which inputs it used. A feature attribution answers that. For one row and one feature, it is that feature's contribution to the difference between that row's prediction and the model's average prediction. The method used here is SHAP (SHapley Additive exPlanations).

Attributions sit on the log scale of survival time, so an attribution is a multiplier on predicted time rather than a number of days. An attribution of +0.3 multiplies that row's predicted survival time by about 1.35, and a negative one shortens it. That conversion follows from the scale and says nothing about this dataset.

Averaging a feature's attributions by size, ignoring sign, says how much that feature moves predictions across the explanation sample, a random subset of the final model's training rows drawn for the attribution computation. A feature high in that average moves predictions a lot, and one low in it barely moves them.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks the strongest features by that average, and Figure 5 shows, for each of those features, which direction it pushed and how much that varied from row to row.

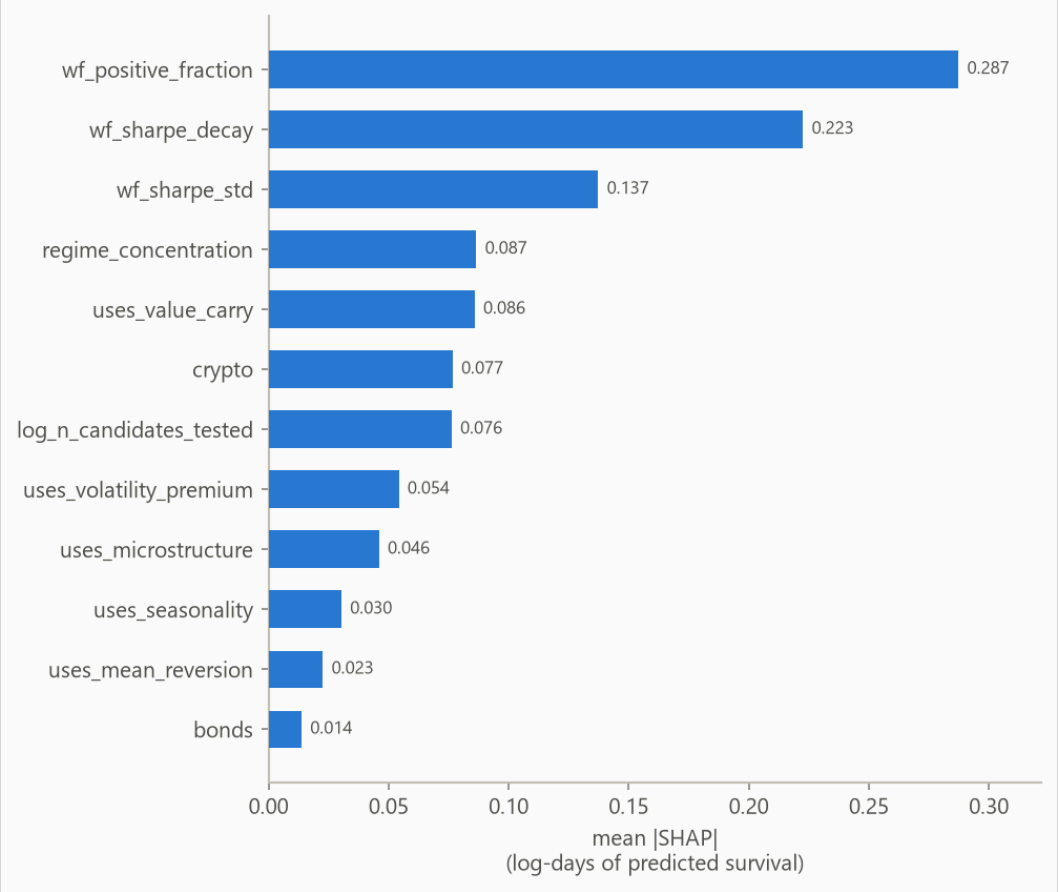

Figure 4. The strongest features, ranked by the average size of their attributions across the explanation sample. Size is taken without regard to sign, so this says how much each feature moves predictions and not which way it moves them; the direction is in Figure 5. The scale is the same log scale of survival time, so a feature averaging 0.10 moves predicted survival time by about a tenth on a typical row. A yes/no flag from a text column keeps its column name in the label. Features below the strongest few are not drawn.

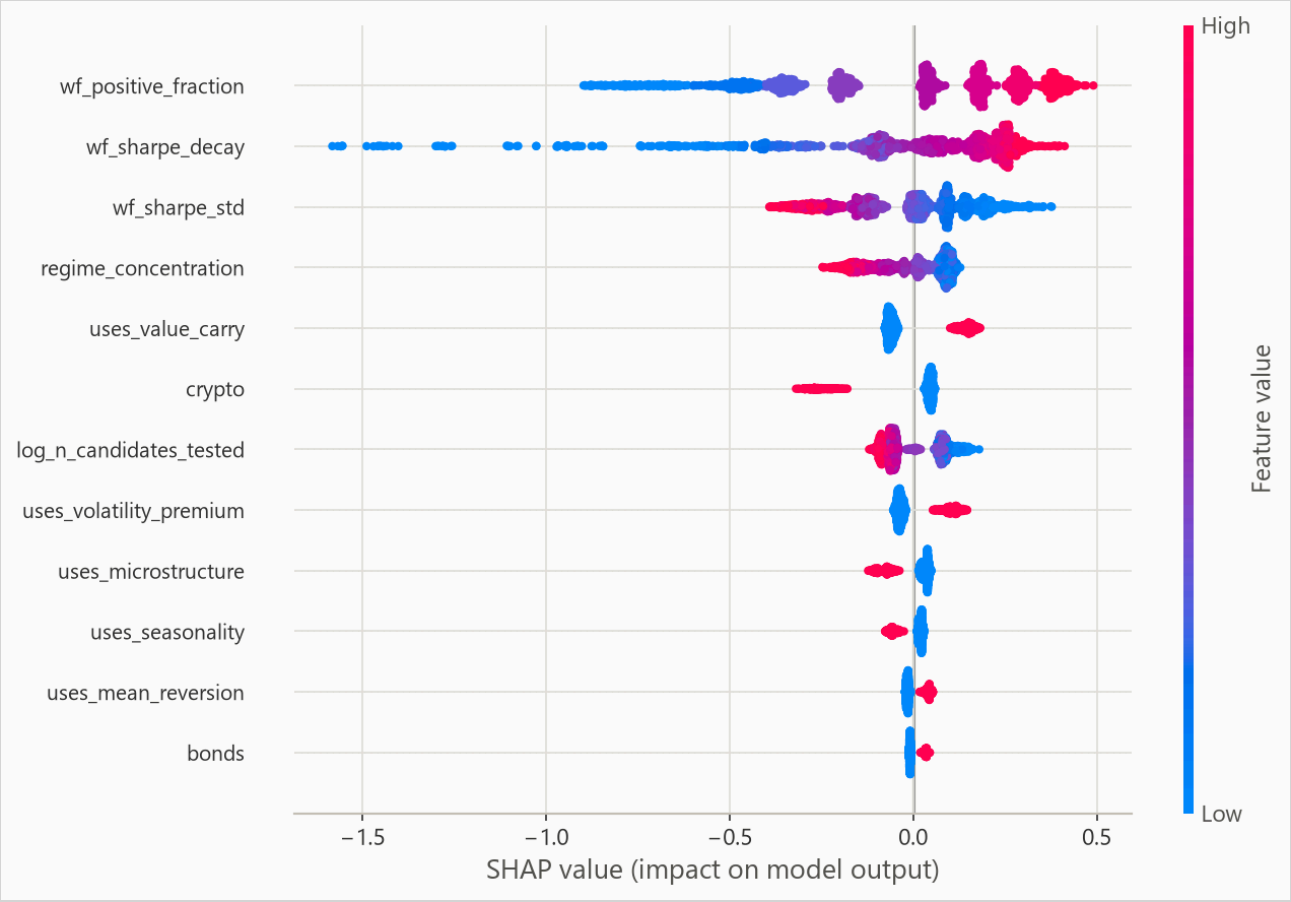

Figure 5. One row per feature, one dot per row of the explanation sample. The dot's position is the attribution that feature earned for that row, so dots left of zero shortened its predicted survival time and dots right of it lengthened. The dot's color is that row's value of the feature, red high and blue low, so a row whose reds sit left of its blues is a feature whose high values shorten survival. How far the dots spread says how much the feature's effect varies from row to row, and a feature pinned at zero did nothing. Single attributions run wider than the averages in Figure 4, since averaging over rows shrinks them. A yes/no flag from a text column keeps its column name in the label and reads the same way, with red meaning the row has that value.

6.2 The Cox baseline

The Cox baseline's drivers are its coefficients, reported as hazard ratios. A hazard ratio is the factor by which one unit of a feature multiplies the hazard and a hazard is the risk of an ending at a given age. The factor is the same at every age, so a ratio above 1 shortens survival. Figure 6 plots the strongest covariates.

Figure 4 describes the boosted model and Figure 6 the Cox baseline, two different models read by two different methods, so they need not name the same features, and neither is a verdict on the data.

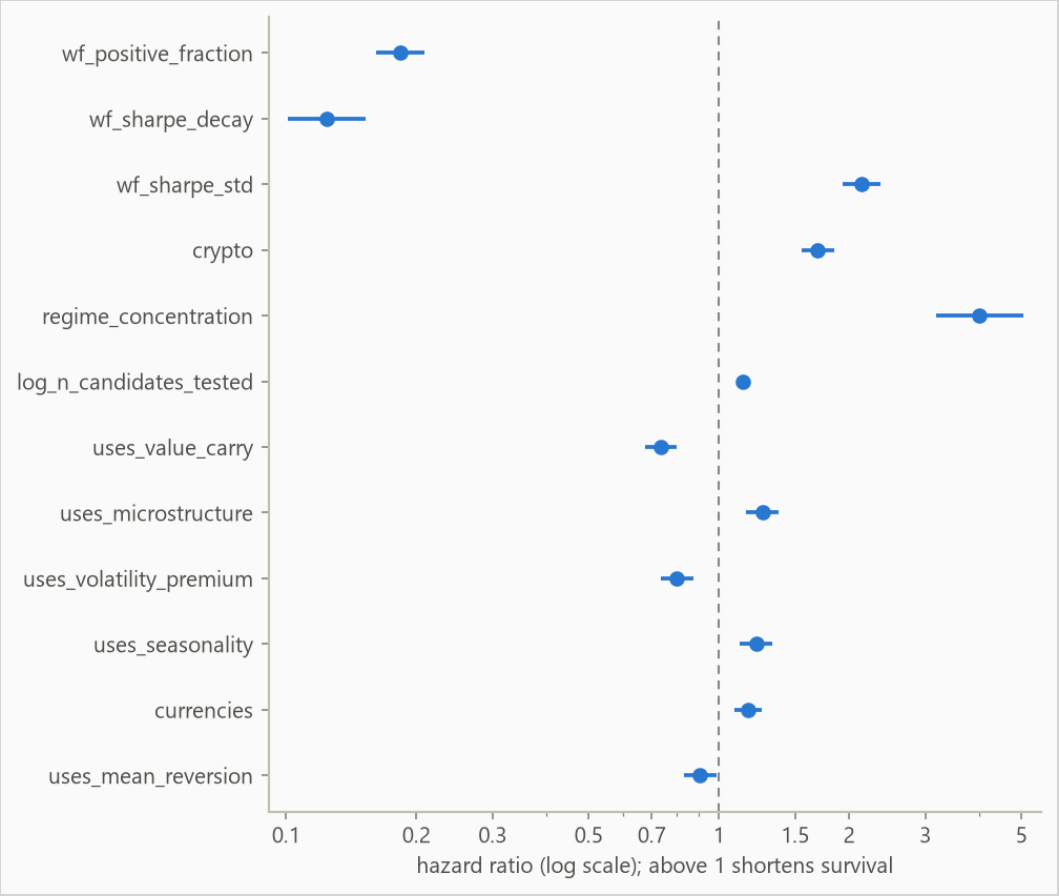

Figure 6. The Cox Proportional Hazards model's strongest covariates with their 95% intervals on a log axis. Dots right of the line shorten survival and left lengthen it. The covariate whose effect the data pins down most firmly sits at the top, and only the strongest few are drawn. Ratios for `asset_class` are relative to bonds, the reference level, chosen alphabetically and given no bar of its own.

7. Interpretation

Analyst notes:

Reading the tie. The boosted model and the Cox baseline land at 0.781 and 0.781 on the fold mean, which is essentially a tie. The synthetic data generator's observable structure is close to additive, which leaves little for trees to find beyond what a penalized linear model in the log-hazard captures. I chose to report the tie rather than adding interactions to the generator until the headline model wins, because a benchmark tuned until it loses is not a benchmark.

The calibration failure worth keeping. An earlier version of this pipeline selected hyperparameters by concordance and looked correct, then lost to a no-skill forecast on 365-day Brier score. Concordance only compares ordering and is invariant to the predictive scale. So the search had settled on a distribution width

far too wide and nothing in the training loop objected. That failure is why selection now uses censored log-likelihood and why every report this pipeline produces grades probability quality separately from ranking.

What the attribution shows, checked against the mechanism. The features the model leans on most are the walk-forward statistics, the record of how a strategy performed across successive validation windows. It is important to note that none of these statistics drive survival time directly in the generator. In the generator, survival time is driven by two hidden quantities. These are how much real edge a strategy has and how much of its validation score was overfitting. The walk-forward statistics are noisy proxies of those two quantities, and the model found them. The generator gives every feature family and every asset class a fixed shift on log survival time, and the model's attribution for each one carries the same sign the generator installed. The walk-forward statistics have no installed shift, but the generator does fix a direction. Walk-forward positive fraction rises with real edge in the generator, so it should push predicted survival up, and it does. Walk-forward Sharpe decay and fold-to-fold Sharpe dispersion both rise with overfitting, so they should push predicted survival down, and they do. Validation Sharpe itself barely registers in the attribution ranking. Past the selection bar it carries more overfitting than edge, so the model gains little from it. The model recovering the installed effects directly, and reaching the two hidden drivers through their proxies, is the evidence this run exists to produce. The pipeline fits real structure where the answer is known, which is the ground for trusting its evaluation on data where the answer is not.

8. Limitations

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. If the rows that drop out of observation are the ones about to end, the survival estimates come out too high.

Analyst notes:

The generator encodes a prior, not evidence, so the concordance here is an upper bound on production performance. Lifetimes are drawn and resampled independently, understating real-book uncertainty, and administrative retirement is modeled as independent censoring, with competing risks left as future work. Every number here comes from one generator seed, so data variance and model variance cannot be separated.

The boosted model gives every row one curve shape. The predictive scale is a single fitted number, so the model runs a row's clock faster or slower but never changes the curve's shape. Per-row width is future work.

Harrell's concordance is biased under heavy censoring, usually upward. It scores only the pairs where the earlier ending is known, and under heavy censoring those pairs over-represent short observed lifetimes. The most censored test window is 39% censored, so read its rows in Table 2 with the most doubt.

9. Reproducing this run

Python 3.11 or later. From the repository root:

```
pip install -r requirements.txt
python -m pytest
python scripts/run_synthetic_pipeline.py
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Generated from `reports/synthetic/metrics.json` by `scripts/run_build_report.py`. 5,000 rows drawn at seed 7, 3,000 out-of-time test rows.